

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

А.В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

ЗАПРОСЫ НА ЯЗЫКЕ SQL:
АГРЕГАТНЫЕ ФУНКЦИИ

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

И.А. Лаврешин
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Освоить использование агрегатных функций и группировки, а также операций над результатами SELECT.

2 Задание и исходные данные

Реализовать запросы г – е, указанные в варианте задания.

Самостоятельно предложить и реализовать запросы с агрегатными функциями, не использованными при выполнении варианта задания.

Самостоятельно предложить и реализовать запросы, демонстрирующие применение каждой из директив, приведенных в п. 4.2., показать различия в получаемых результатах при выполнении теоретико-множественных операций и операций над мультимножествами.

3 Ход выполнения задания

г) Средняя численность подразделений. Под “численностью подразделения” принимаем сумму поля количество_единиц по всем строкам таблицы штатное_расписания для данного ид_подразделения. Внутренний запрос группирует штат по подразделениям и считает SUM, внешний — усредняет эти суммы функцией AVG и округляет результат (ROUND) для читаемой таблицы результатов.

д) Инженеры с более чем пятью детьми. Строим связь сотрудник—дети, отбираем сотрудников, для которых существует приём на должность с категорией “инженер” (EXISTS по прием и должность). GROUP BY по сотруднику, HAVING COUNT(детей) > 5 сортируем по ФИО.

е) Подразделения, где техников больше, чем инженеров. Берём приёмы по подразделениям, к каждому присоединяем должность. В одной группе по подразделению считаем число DISTINCT сотрудников с категорией «техник» и отдельно — с категорией “инженер” (CASE внутри COUNT(DISTINCT ...)). Условие на группы задаётся в HAVING.

На рисунках 1 – 4 представлены результаты работы программы и её листинг.

```

1 SELECT
2   ROUND(AVG(per_dept.сумма_единиц)::numeric, 4) AS средняя_численность_подразделений
3 FROM (
4   SELECT
5     шр.ид_подразделения,
6     SUM(шр.количество_единиц) AS сумма_единиц
7   FROM штатное_расписание шр
8   GROUP BY шр.ид_подразделения
9 ) AS per_dept;
10
11 SELECT
12   с.ид,
13   с.фio,
14   COUNT(д.ид) AS число_детей
15 FROM сотрудник с
16   INNER JOIN дети д ON д.ид_сотрудника = с.ид
17 WHERE EXISTS (
18   SELECT 1
19   FROM прием пр
20     INNER JOIN должность дл ON дл.ид = пр.ид_должности
21   WHERE пр.ид_сотрудника = с.ид
22     AND дл.категория = 'инженер'
23 )
24 GROUP BY с.ид, с.фio
25 HAVING COUNT(д.ид) > 5
26 ORDER BY с.фio;
27
28 SELECT
29   пд.ид,
30   пд.наименование AS подразделение,
31   COUNT(DISTINCT CASE WHEN дл.категория = 'техник' THEN пр.ид_сотрудника END) AS техников,
32   COUNT(DISTINCT CASE WHEN дл.категория = 'инженер' THEN пр.ид_сотрудника END) AS инженеров
33 FROM подразделение пд
34   INNER JOIN прием пр ON пр.ид_подразделения = пд.ид
35   INNER JOIN должность дл ON дл.ид = пр.ид_должности
36 GROUP BY пд.ид, пд.наименование
37 HAVING COUNT(DISTINCT CASE WHEN дл.категория = 'техник' THEN пр.ид_сотрудника END)
38 > COUNT(DISTINCT CASE WHEN дл.категория = 'инженер' THEN пр.ид_сотрудника END)
39 ORDER BY пд.ид;

```

Рисунок 1 – Запросы г-е

	средняя_численность_подразделений numeric
1	5.3333

Рисунок 2 – Результаты запроса г

	ид [PK] integer	фio character varying (200)	число_детей bigint
1	6	Большая Семья Инженер	7

Рисунок 3 – Результаты запроса д

	ид [PK] integer	подразделение character varying (200)	техников bigint	инженеров bigint
1	2	Цех механической обработки	3	1
2	3	Отдел главного конструктора	4	3

Рисунок 4 – Результаты запроса е

В основных пунктах г) – е) для моего варианта MIN и MAX не использовались, поэтому дополнительно выполняем два запроса с минимумом и максимумом. По каждому подразделению: MIN и MAX даты приёма среди

строк таблицы прием. По каждой должности из справочника: минимальное и максимальное количество_единиц в одной строке штатное_расписание для этой должности. Результаты представлены на рисунках 5 – 6.

```

1  SELECT
2      пд.ид,
3      пд.наименование AS подразделение,
4      MIN(пр.дата_приема) AS первый_прием,
5      MAX(пр.дата_приема) AS последний_прием
6  FROM подразделение пд
7      INNER JOIN прием пр ON пр.ид_подразделения = пд.ид
8  GROUP BY пд.ид, пд.наименование
9  ORDER BY пд.ид;
10
11 SELECT
12     д.ид,
13     д.наименование AS должность,
14     MIN(шр.количество_единиц) AS мин_единиц_в_строке,
15     MAX(шр.количество_единиц) AS макс_единиц_в_строке
16  FROM должность д
17      INNER JOIN штатное_расписание шр ON шр.ид_должности = д.ид
18  GROUP BY д.ид, д.наименование
19  ORDER BY д.ид;

```

Рисунок 5 – Запросы для max и min

	ид [PK] integer	должность character varying (150)	мин_единиц_в_строке smallint	макс_единиц_в_строке smallint
1	1	Инженер-конструктор 1 кат.	2	3
2	2	Инженер-технолог	2	2
3	3	Техник	1	5

Рисунок 6 – Результаты для max и min

Для UNION и UNION ALL в работе задаются два запроса к таблице сотрудников с разными условиями по фамилии, что показано на рисунке 7: строки попадающих в первую выборку, и второй — строки второй выборки.

```

1  SELECT с.фio
2  FROM сотрудник с
3  WHERE с.фio LIKE 'И%'
4  UNION
5  SELECT с2.фio
6  FROM сотрудник с2
7  WHERE с2.фio LIKE '%ОВ%'
8  ORDER BY 1;
9
10 SELECT с.фio
11 FROM сотрудник с
12 WHERE с.фio LIKE 'И%'
13 UNION ALL
14 SELECT с2.фio
15 FROM сотрудник с2
16 WHERE с2.фio LIKE '%ОВ%'
17 ORDER BY 1;|

```

Рисунок 7 – Запрос для UNION и UNION ALL

У одного и того же сотрудника оба условия могут выполняться одновременно — тогда строка будет “кандидатом” для обеих частей объединения. При UNION сервер считает объединённые строки множеством: совпадающее ФИО в результате представляется один раз. Результат представлен на рисунке 8.

	фio character varying (200)	
1	Васильев Тимур Тимурович	
2	Иванов Иван Иванович	
3	Козлова Анна Викторовна	
4	Николаев Олег Олегович	
5	Петрова Мария Сергеевна	
6	Сидоров Пётр Алексеевич	
7	Только техники (руководитель цеха)	

Рисунок 8 – Результаты для UNION

При UNION ALL объединение ведётся как у мультимножества, поэтому та же строка может прийти и из первого, и из второго SELECT и оказаться в результате дважды. Результат представлен на рисунке 9.

	фio character varying (200)	🔒
1	Васильев Тимур Тимурович	
2	Иванов Иван Иванович	
3	Иванов Иван Иванович	
4	Козлова Анна Викторовна	
5	Николаев Олег Олегович	
6	Петрова Мария Сергеевна	
7	Сидоров Пётр Алексеевич	
8	Только техники (руководитель цеха)	

Рисунок 9 – Результаты для UNION ALL

Операция INTERSECT задаёт пересечение двух множеств строк. В использованном примере, листинг которого представлен на рисунке 10, в первое множество входят идентификаторы сотрудников, у которых в таблице прием есть строка для подразделения с кодом 2, во второе — те же ид_сотрудника, но для подразделения 4.

```

1  SELECT пр.ид_сотрудника
2  FROM прием пр
3  WHERE пр.ид_подразделения = 2
4  INTERSECT
5  SELECT пр2.ид_сотрудника
6  FROM прием пр2
7  WHERE пр2.ид_подразделения = 4
8  ORDER BY 1;|

```

Рисунок 10 – Операция INTERSECT

В результат попадают только те работники, для которых в данных одновременно присутствуют записи приёма в обоих подразделениях, порядок сортировки по ид позволяет удобно сверять вывод при оформлении отчёта. Результат представлен на рисунке 11.


```

1  SELECT пр.ид_сотрудника
2  FROM прием пр
3  WHERE пр.ид_подразделения = 2
4  INTERSECT
5  SELECT пр2.ид_сотрудника
6  FROM прием пр2
7  WHERE пр2.ид_подразделения = 4
8  ORDER BY 1;

```

Рисунок 11 – Результат операции

Для операции EXCEPT («разность») формируется множество сотрудников, фигурирующих среди действующих или исторических записей прием, из которого убираются все ид_сотрудника, встретившиеся в таблице увольнение. То есть в результирующей таблице, представленной на рисунках 12 – 13, остаются сотрудники с приёмом, по которым нет строки об увольнении.

```

SELECT с.фio
FROM сотрудник с
WHERE с.фio LIKE 'И%'
UNION
SELECT с2.фio
FROM сотрудник с2
SELECT DISTINCT пр.ид_сотрудника
FROM прием пр
EXCEPT
SELECT у.ид_сотрудника
FROM увольнение у
ORDER BY 1;

```

Рисунок 12 – Операция EXCEPT

	ид_сотрудника integer	
1		1
2		2
3		3
4		4
5		6
6		7
7		8
8		9
9		10
10		11

Рисунок 13 – Результат операции EXCEPT

4 Вывод

Выполнены агрегатные запросы с группировкой и HAVING, отработаны MIN/MAX на связанных с приёмами и штатом данных, а также продемонстрированы UNION/UNION ALL, INTERSECT, EXCEPT и при необходимости варианты ALL.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
-- 01_queries_gde.sql
-- з) Средняя численность подразделений.
-- Численность подразделения = сумма штатных единиц по строкам
штатного_расписания;
-- «средняя численность» = среднее этих сумм по всем подразделениям (AVG
по группам).
```

```
SELECT
    ROUND(AVG(per_dept.сумма_единиц)::numeric, 4) AS
средняя_численность_подразделений
FROM (
    SELECT
        шр.ид_подразделения,
        SUM(шр.количество_единиц) AS сумма_единиц
    FROM штатное_расписание шр
    GROUP BY шр.ид_подразделения
) AS per_dept;
```

```
-- д) Инженеры, у которых более пяти детей.
-- Инженер: у сотрудника есть приём на должность с категорией
«инженер».
```

```
SELECT
    с.ид,
    с.фio,
    COUNT(д.ид) AS число_детей
FROM сотрудник с
    INNER JOIN дети д ON д.ид_сотрудника = с.ид
WHERE EXISTS (
    SELECT 1
```

```

FROM прием пр
    INNER JOIN должность дл ON дл.ид = пр.ид_должности
WHERE пр.ид_сотрудника = с.ид
    AND дл.категория = 'инженер'
)
GROUP BY с.ид, с.фio
HAVING COUNT(д.ид) > 5
ORDER BY с.фio;

```

-- е) Подразделения, в которых количество техников превышает количество инженеров.

-- Считаем различных сотрудников по текущим приёмам (таблица прием) и категории должности.

```

SELECT
    пд.ид,
    пд.наименование AS подразделение,
    COUNT(DISTINCT CASE WHEN дл.категория = 'техник' THEN
пр.ид_сотрудника END) AS техников,
    COUNT(DISTINCT CASE WHEN дл.категория = 'инженер' THEN
пр.ид_сотрудника END) AS инженеров
FROM подразделение пд
    INNER JOIN прием пр ON пр.ид_подразделения = пд.ид
    INNER JOIN должность дл ON дл.ид = пр.ид_должности
GROUP BY пд.ид, пд.наименование
HAVING COUNT(DISTINCT CASE WHEN дл.категория = 'техник' THEN
пр.ид_сотрудника END)
    > COUNT(DISTINCT CASE WHEN дл.категория = 'инженер' THEN
пр.ид_сотрудника END)
ORDER BY пд.ид;

```

--02_extra_aggregates.sql

-- 1) MIN / MAX: по подразделению — первая и последняя дата приёма.

SELECT

 пд.ид,

 пд.наименование AS подразделение,

 MIN(пр.дата_приема) AS первый_прием,

 MAX(пр.дата_приема) AS последний_прием

FROM подразделение пд

 INNER JOIN прием пр ON пр.ид_подразделения = пд.ид

GROUP BY пд.ид, пд.наименование

ORDER BY пд.ид;

-- 2) MIN / MAX: по должности — минимальное и максимальное число штатных единиц в одной строке штатного расписания.

SELECT

 д.ид,

 д.наименование AS должность,

 MIN(шр.количество_единиц) AS мин_единиц_в_строке,

 MAX(шр.количество_единиц) AS макс_единиц_в_строке

FROM должность д

 INNER JOIN штатное_расписание шр ON шр.ид_должности = д.ид

GROUP BY д.ид, д.наименование

ORDER BY д.ид;

--03_set_operations_4_2.sql

-- ===== UNION: объединение как над множеством (дубликаты строк схлопываются) =====

SELECT с.фio

FROM сотрудник с

WHERE с.фio LIKE 'И%'

```

UNION
SELECT с2.фio
FROM сотрудник с2
WHERE с2.фio LIKE '%ОВ%'
ORDER BY 1;

```

-- ===== UNION ALL: объединение мультимножеств (кратности суммируются) =====

```

SELECT с.фio
FROM сотрудник с
WHERE с.фio LIKE 'И%'
UNION ALL
SELECT с2.фio
FROM сотрудник с2
WHERE с2.фio LIKE '%ОВ%'
ORDER BY 1;

```

-- Сравните число строк двух запросов выше (у «Иванов ...» оба условия — UNION даст 1 строку, UNION ALL — 2).

-- ===== INTERSECT: пересечение множеств (уникальные общие строки) =====

```

SELECT пр.ид_сотрудника
FROM прием пр
WHERE пр.ид_подразделения = 2
INTERSECT
SELECT пр2.ид_сотрудника
FROM прием пр2
WHERE пр2.ид_подразделения = 4
ORDER BY 1;

```

-- ===== *EXCEPT: разность множеств* =====

```
SELECT DISTINCT пр.ид_сотрудника
FROM прием пр
EXCEPT
SELECT у.ид_сотрудника
FROM увольнение у
ORDER BY 1;
```

-- ===== *INTERSECT ALL / EXCEPT ALL (мультимножества; PostgreSQL 15+)* =====

-- *Правила кратностей — как в п. 4.2 методички; в MS SQL для INTERSECT/EXCEPT с ALL ограничения.*

```
SELECT a
FROM (VALUES (1), (1), (2)) AS r(a)
INTERSECT ALL
SELECT a
FROM (VALUES (1), (1), (1)) AS s(a);
```

```
SELECT a
FROM (VALUES (1), (1), (2)) AS r2(a)
EXCEPT ALL
SELECT a
FROM (VALUES (1)) AS s2(a);
```